

PUBLIC TECHNICAL REPORT

UVM Verification of a P4 Adder and a Windowed Register File

Coverage-driven verification at RTL and after synthesis

Erfan Taheri

SoC Verification Strategies

Contents

1	Introduction	1
2	P4 adder verification	2
2.1	Verification goal and checklist	2
2.2	RTL behavior and verification focus	2
2.3	UVM environment and timing model	3
2.4	Stimulus methodology	3
2.5	Functional coverage groups	4
2.6	Coverage closure and results	5
3	Windowed register-file verification	6
3.1	Verification goal and checklist	6
3.2	RTL behavior and address mapping	6
3.3	UVM environment	8
3.4	Independent cycle-accurate reference model	8
3.5	Stimulus and state-space exploration	9
3.6	Functional coverage of the FSM and counters	9
3.7	RAL boundary	10
3.8	Results and coverage closure	11
3.9	Code coverage exposes the window-overflow defect	11
3.10	Post-synthesis findings	12
4	Conclusion	12
5	Learning resources	13

1 Introduction

This report presents two Universal Verification Methodology (UVM) environments developed for a registered 32-bit P4 adder and a 64-bit windowed register file. The common objective was to create self-checking, coverage-driven environments in which an independent model decides correctness and coverage reports guide closure. The two environments share an agent, monitor, scoreboard, sequence layering, and functional coverage, but their timing models and verification targets are different.

Table 1: *RTL scope and verification focus.*

Block	RTL summary	Verification focus
P4 adder	Registered 32-bit sparse-tree carry-select adder divided into eight 4-bit carry blocks, with a fixed two-cycle input-to-output latency.	Pipeline alignment, arithmetic correctness, block propagate and generate behavior, full-width carry ripple, prefix-tree expression terms, and reset behavior.
Windowed register file	Eight overlapping windows, eight fixed global registers, 136 physical 64-bit registers, and a window-management state machine controlled by <code>CALL</code> and <code>RET</code> .	Address translation, adjacent-window overlap, registered reads, CWP/SWP transitions, save/restore counters, spill/fill boundaries, fixed-map globals, and RAL access.

The adder verification concentrates on a pipelined arithmetic datapath and the internal conditions that activate its carry tree. The register-file verification is stateful and cycle accurate: each monitored transaction represents one clock, and the reference model advances the physical array and window-management state in lockstep with the RTL.

2 P4 adder verification

2.1 Verification goal and checklist

The verification goal is to prove that the registered adder returns the correct 33-bit arithmetic result at the specified latency and that stimulus exercises the carry architecture, not only representative operand values.

Table 2: *P4 specification checklist.*

Requirement	Acceptance criterion
Arithmetic result	For every checked transaction, $\{C_{out}, S\} = A + B + C_{in}$.
Pipeline timing	The result observed at cycle $T + 2$ is paired with the operands accepted at cycle T .
Reset	Registered datapath state clears, and the monitor discards transactions invalidated by reset.
Carry architecture	Every 4-bit block exercises propagate, generate, incoming-carry, and actual ripple behavior.
Critical cases	Arithmetic corners, carry input/output, full-width propagation, and non-masking prefix-tree terms are exercised.
Implementation reuse	The same checker and functional-coverage model run on the mapped netlist, and the timing constraint is met.

2.2 RTL behavior and verification focus

The DUT combines a sparse parallel-prefix carry generator with carry-select sum blocks. The 32-bit datapath is partitioned into eight 4-bit blocks. Input and output registers create a fixed two-cycle latency, so the monitor must associate each observed result with the operands accepted two cycles earlier.

The scoreboard computes the expected 33-bit result independently as

$$\{C_{out}, S\} = A + B + C_{in}.$$

The arithmetic operator does not reproduce the RTL carry tree. A defect in the tree therefore cannot be copied into the expected result. Four-state comparison is used so that an undefined DUT output is treated as a mismatch.

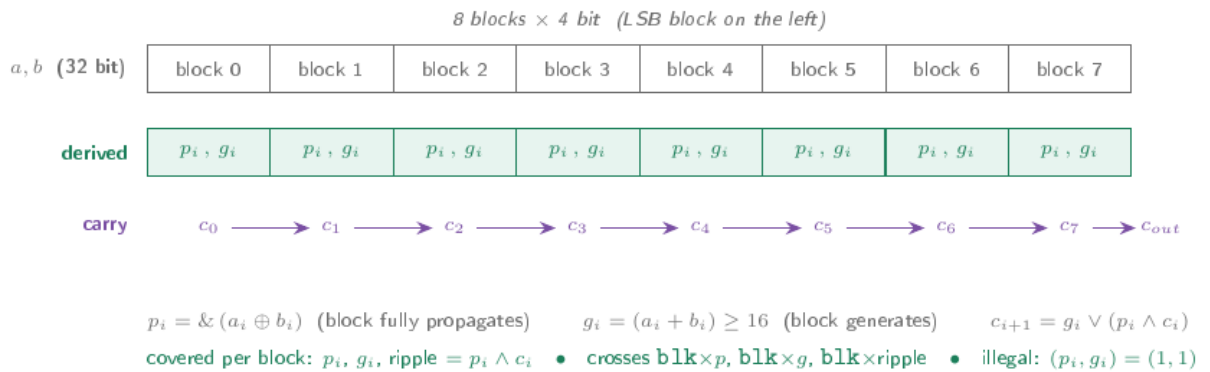


Figure 1: *P4 block structure and the architecture-aware quantities derived for coverage.*

2.3 UVM environment and timing model

The sequencer and driver apply one addition per transaction through a clocking-block interface. The monitor maintains a two-entry pipeline of operands and publishes only complete $(A, B, C_{in}, S, C_{out})$ transactions. The scoreboard therefore receives aligned data and remains independent of the DUT latency. The same monitored stream feeds the functional-coverage collector.

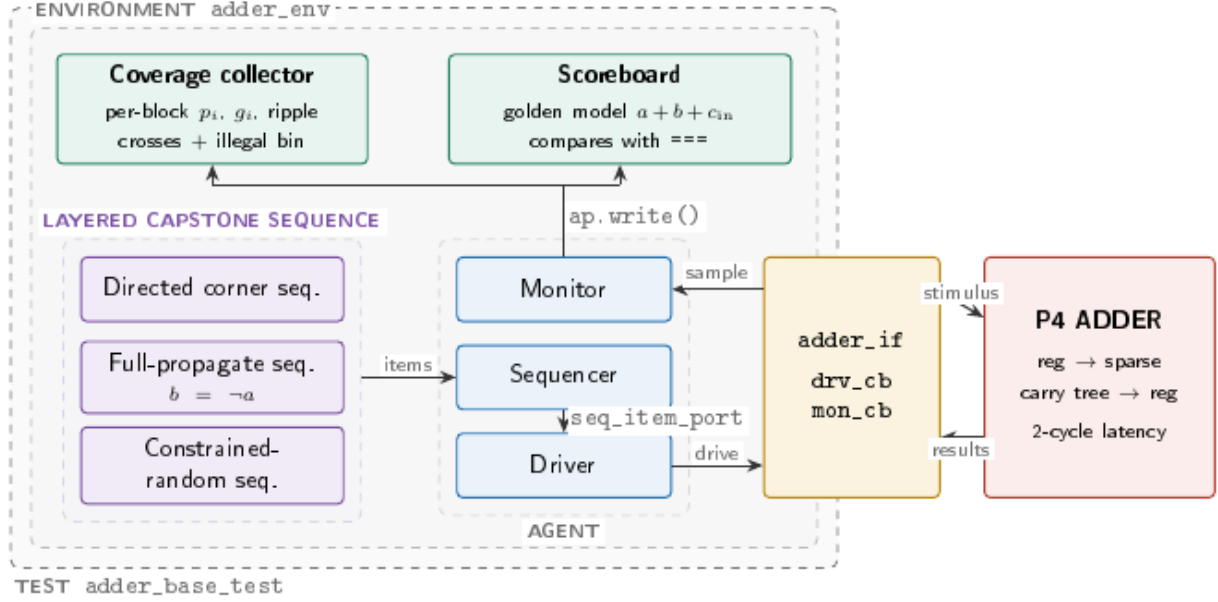


Figure 2: *P4 UVM environment. Operand and result alignment is completed in the monitor before analysis broadcast.*

2.4 Stimulus methodology

The layered regression combines several sequence families:

- directed arithmetic corners for zero, maximum values, carry input, and carry output;
- constrained-random operands weighted toward zero, all ones, and mid-range values;
- a full-propagation sequence with $B = \neg A$, forcing every bit to propagate and allowing a carry to cross all eight blocks;
- directed prefix-tree vectors that remove masking from individual propagate/generate terms;
- reset stimulus added during closure so the registered wrapper and all reset coverage bins are exercised.

The full-propagation case is required explicitly because an unconstrained 32-bit random test reaches $A \oplus B = \text{all ones}$ with probability 2^{-32} .

Selected stimulus: full-width carry propagation

```

add_prop_seq.svh · selected excerpt

task body();
  repeat (n) begin
    adder_seq_item req = adder_seq_item::type_id::create("req");

    start_item(req);
    assert(req.randomize() with { (a ^ b) == '1; });
    finish_item(req);
  end
endtask

// a ^ b == '1 makes every bit propagate an incoming carry.

```

Figure 3: Selected full-propagation sequence.

Coverage closure: reset sequence exercises the remaining reset transition

```

add_reset_seq.svh · selected excerpt

class adder_reset_seq extends uvm_sequence #(adder_seq_item);
  task body();
    adder_seq_item req;

    req = adder_seq_item::type_id::create("reset_req");
    start_item(req);
    req.op = ADDER_RESET;
    req.reset_cycles = 2;
    finish_item(req);
  endtask
endclass

// Added during closure to complete reset-related code coverage.

```

Figure 4: Reset sequence added to close reset and registered-wrapper coverage.

2.5 Functional coverage groups

Operand-value bins alone do not describe whether the carry architecture was exercised. The coverage model instead derives, for every 4-bit block, the group propagate p_i , group generate g_i , carry entering the block, and the ripple condition $p_i \wedge c_i$. A covergroup with an explicit sample function is sampled once for every block.

Table 3: *P4 functional coverage model.*

Coverpoint or cross	Verification question
Block index	Was every 4-bit carry block sampled?
Propagate and generate	Was each block observed in both architectural conditions?
Carry ripple	Did a carry actually cross a block boundary?
Block by propagate/generate	Were propagate and generate reached at every block position?
Block by ripple	Did a carry ripple through every one of the eight blocks?
Carry input, carry output, full propagation	Were top-level carry behavior and the critical full-width case exercised?
Illegal $(p, g) = (1, 1)$	Did the coverage derivation ever report an impossible state?

Selected coverage model: each carry block must propagate, generate, and ripple

```

add_cover.svh · selected excerpt

covergroup cg_block with function sample
  (int id, bit p, bit g, bit carry_in);

  cp_block: coverpoint id { bins blk[] = {[0:NB-1]}; }
  cp_p:      coverpoint p;
  cp_g:      coverpoint g;
  cp_ripple: coverpoint (p & carry_in) {
    bins yes = {1}; bins no = {0};
  }

  x_block_p: cross cp_block, cp_p;
  x_block_g: cross cp_block, cp_g;
  x_block_ripple: cross cp_block, cp_ripple;

  impossible_pg: cross cp_p, cp_g {
    illegal_bins both = binsof(cp_p) intersect {1}
                  && binsof(cp_g) intersect {1};
  }
endgroup

```

Figure 5: *Selected P4 coverpoints, per-block crosses, and the impossible-state illegal bin.*

2.6 Coverage closure and results

Coverage closure followed the reports rather than a larger random transaction count. Directed vectors exercised non-masking combinations in the prefix equations. A reset sequence closed the remaining reset-related bins and transitions. The final RTL report records complete functional, statement, branch, expression, toggle, and assertion coverage.

Two testbench issues were also found while closing coverage. Unsigned operand-width types were required for high-value coverage ranges, avoiding signed truncation of values such as `32'hF0000000`. The test also needed a drain period before dropping its UVM objection so the final registered transactions could leave the DUT and monitor pipeline.

The same UVM environment was reused against the mapped netlist with back-annotated timing. The functional model again reached 117 of 117 bins. A deliberate fault-injection run also produced scoreboard mismatches and returned to zero errors when the fault was removed, confirming that the checker was capable of rejecting an incorrect result.

Table 4: *P4 verification and synthesis results.*

Metric	Result
Scoreboard	601 checks, 0 mismatches
Functional coverage	100% (117/117 bins)
Statement coverage	100%
Branch coverage	100%
Expression coverage	100%
Toggle coverage	100%
Assertion coverage	100% (14/14)
Filtered overall RTL coverage	100%
Post-synthesis functional coverage	100% (117/117 bins)
Post-synthesis filtered coverage	99.95%
Post-synthesis toggle coverage	99.86% (761/762)
Clock constraint	2.00 ns
Worst timing slack	0.01 ns, met

3 Windowed register-file verification

3.1 Verification goal and checklist

The verification goal is to prove the logical register view, moving window state, and registered read behavior on every clock, including the overflow and underflow boundaries.

Table 5: *Windowed register-file specification checklist.*

Requirement	Acceptance criterion
Reset state	Clear the 136-entry array; initialize CWP and SWP to 0, CANSERVE to 7, CANRESTORE to 0, and both boundary flags low.
Address translation	Map IN, LOCAL, and OUT through CWP with modulo-128 wrap-around; map globals to fixed physical registers 128–135.
Window overlap	After CALL, caller OUT and callee IN resolve to the same eight physical registers.
Data timing	Verify both registered read ports, one-cycle latency, read-before-write behavior, and defined idle cycles.
Control priority	Verify CALL over RET over normal access, and state hold when disabled.
Window state	Verify CWP/SWP movement, every save/restore counter value, and single-cycle SPILL/FILL pulses.
Boundary semantics	Compare overflow and underflow behavior with the expected new-window allocation and identify any mismatch.
RAL and netlist reuse	Backdoor-check the eight fixed-map globals and reuse the environment against the mapped netlist.

3.2 RTL behavior and address mapping

The design exposes 32 logical 64-bit registers: eight input, eight local, eight output, and eight global registers. Eight windows occupy 128 physical registers, while the global block occupies fixed physical

locations 128 through 135. Each window spans 24 physical locations, and a `CALL` advances the current window pointer by 16. The caller's output block and the callee's input block therefore share the same eight physical registers.

The window-management state contains the current and saved window pointers (CWP and SWP), plus `CANSAVE` and `CANRESTORE`. When the available-window pool is exhausted, `CALL` raises `SPILL`; the reverse boundary raises `FILL`.

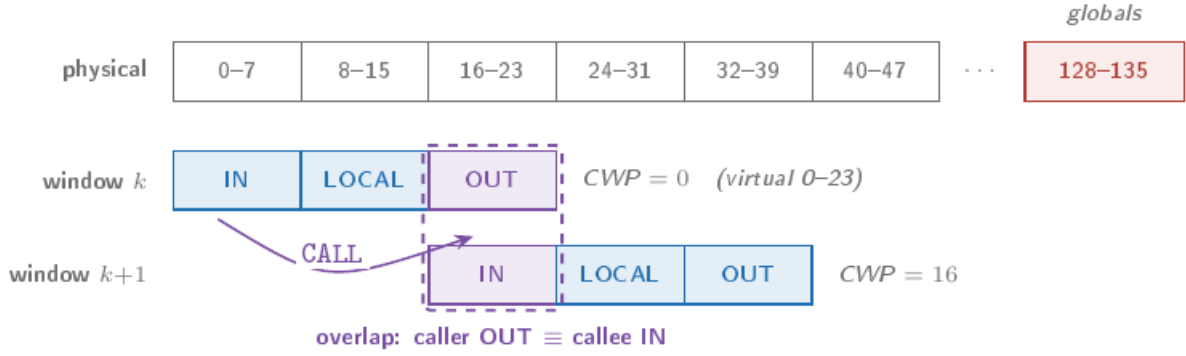


Figure 6: Logical-to-physical mapping and the overlap between adjacent windows.

Every clock is meaningful for this DUT, so a sequence item represents one complete cycle: reset, enable, call, return, read/write controls, logical addresses, input data, and the observed registered responses. The driver uses non-blocking item retrieval and drives an explicit idle when no item is pending. This prevents one-shot controls such as `CALL`, `RET`, and write enable from remaining asserted across an idle cycle.

The diagram illustrates the architecture of the `rf_env` environment, which is used to test the `RAL uvm_reg_block`. The environment is divided into several functional blocks:

- ENVIRONMENT `rf_env`**: The overall container for the test environment.
- SCOREBOARD**: A dashed box containing:
 - Coverage collector**: Collects coverage data (op, CWP, CANSAVE/RESTORE, SPILL/FILL, blocks, cross) and sends it to the **Monitor**.
 - Reference model**: Contains `regs[136]`, `CWP`, `SWP`, `CANSAVE`, and `CANRESTORE`. It receives data from the **Monitor** and sends it to the **Comparator**.
 - Comparator**: Compares the reference model with the **DUT** (OUT1/OUT2, SPILL, FILL (==)).
- LAYERED CAPSTONE SEQUENCE**: A sequence of test steps:
 - Reset
 - Write / read-back
 - Overlap (out $\equiv$ in)
 - Call / Ret boundary
 - Globals persistence
 - Constrained-random
- AGENT**: A dashed box containing:
 - Monitor**: 1-cycle align, receives `items` from the sequence and sends `sample` to the **Sequencer**.
 - Sequencer**: Receives `seq_item_port` from the **Monitor** and sends `drive` to the **Driver**.
 - Driver**: Drives the **DUT** (drives idle).
- DUT (Device Under Test)**: A dashed box containing:
 - rf_if**: Interface block with `drv_cb` and `mon_cb`. It receives `stimulus` from the **Sequencer** and sends `responses` back.
 - REGISTERS [0:135]**: 128 windowed + 8 global registers.
 - window FSM**: Controls `CWP`, `SWP`, and `CANSAVE`.
- RAL uvm_reg_block**: A block that provides 8 global registers (fixed map $\rightarrow$ phys. 128-135) and is connected to the **DUT** via a **backdoor**.

The diagram also shows the test configuration at the bottom: `TEST rf_capstone_test / rf_spill_bug_test`.

3.4 Independent cycle-accurate reference model

- the 136-entry physical register array;
- virtual-to-physical translation with wrap-around for the 128 windowed locations and fixed mapping for the eight globals;
- CWP, SWP, CANS_{AVE}, and CAN_{RESTORE};
- registered outputs and the SPILL/FILL pulse behavior;
- read-before-write behavior when the same location is read and written in one cycle;
- the priority of CALL/RET over an access in the same cycle;
- reset behavior, including the fact that the RTL resets the array and state but does not reset the output registers.

8

3.5 Stimulus and state-space exploration

The layered capstone sequence combines write/read-back, adjacent-window overlap, repeated calls and returns, global-register persistence, constrained-random traffic, and a coverage-closure sweep. The overlap sequence writes unique values through the caller’s output registers, performs a **CALL**, then reads the callee’s input registers. Any translation error produces shifted or incorrect values.

The call/return sequence deliberately exceeds both boundaries: seven successful allocations are followed by additional calls that must pulse **SPILL**, then extra returns reach **FILL**. The sweep sequence visits every CWP position and performs idle and access operations at each window before unwinding.

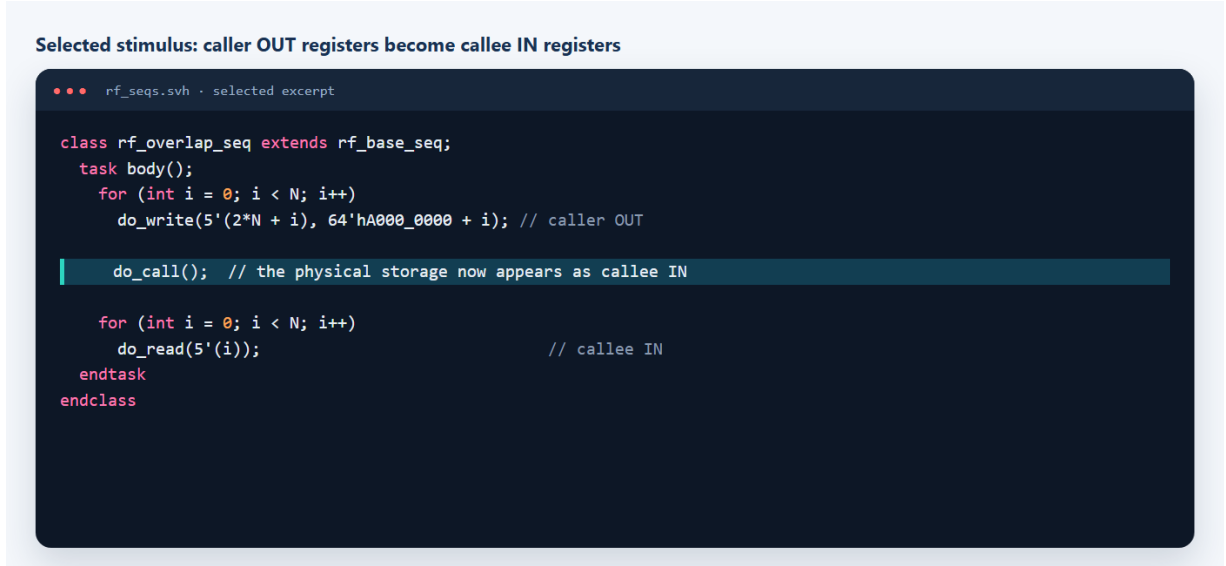


Figure 8: Selected adjacent-window overlap sequence.

3.6 Functional coverage of the FSM and counters

The functional coverage group targets the internal window-management state rather than arbitrary data values. It samples eight coverage quantities:

Table 6: Register-file functional coverage model.

Coverpoint or cross	Coverage intent
Operation	Reset, call, return, normal access, and idle.
CWP	All eight current-window positions: 0, 16, 32, 48, 64, 80, 96, and 112.
CANSAVE	Every occupancy value from 0 through 7.
CANRESTORE	Every occupancy value from 0 through 7.
SPILL and FILL	Both inactive and asserted values for each boundary event.
Addressed block	Input, local, output, global, and no-access cycles.
Operation by CWP	Each reachable operation at every reachable window position.

Window-state coverage and the ignored operation/CWP combinations

```

covergroup cg with function sample(
    int op_i, int cwp, int cansave, int canrestore,
    bit spill, bit fill, int blk_i);

    cp_op:      coverpoint op_i { bins reset={0}; bins call={1};
                                bins ret={2}; bins access={3}; bins idle={4}; }
    cp_cwp:     coverpoint cwp   { bins w[] = {0,16,32,48,64,80,96,112}; }
    cp_cansave: coverpoint cansave { bins v[] = {[0:F-1]}; }
    cp_canrestore: coverpoint canrestore { bins v[] = {[0:F-1]}; }
    cp_spill:    coverpoint spill;
    cp_fill:     coverpoint fill;
    cp_blk:      coverpoint blk_i { bins none={-1}; bins in={0}; bins loc={1};
                                bins out={2}; bins glob={3}; }

    x_op_cwp: cross cp_op, cp_cwp {
        ignore_bins reset_only_at_0 =
            binsof(cp_op) intersect {0} && !binsof(cp_cwp) intersect {0};
        ignore_bins call_never_wraps =
            binsof(cp_op) intersect {1} && binsof(cp_cwp) intersect {0};
        ignore_bins ret_never_at_top =
            binsof(cp_op) intersect {2} && binsof(cp_cwp) intersect {112};
    }
endgroup

```

Seven reset/nonzero-CWP bins are structurally impossible. The CALL/CWP=0 and RET/CWP=112 exclusions instead reflect the implemented boundary behavior tied to the defect.

Figure 9: Selected register-file coverpoints and the operation-by-window cross.

The current covergroup contains three `ignore_bins` declarations that remove nine operation-by-CWP combinations from the denominator. Seven are structurally impossible because reset forces CWP to zero. The remaining two are CALL sampled at CWP 0 and RET sampled at CWP 112. Those two are absent under the implemented spill/fill behavior and therefore relate to the boundary defect rather than to an inherent specification constraint. The preserved reports contain only the resulting model and show 69 of 69 bins covered; no pre-exclusion report is available to support a separate “initial” percentage. Thus, 69 of 69 describes the reduced functional model. The two RTL statement and branch misses remained visible in code coverage and exposed the same boundary problem.

3.7 RAL boundary

UVM RAL assumes a fixed logical address-to-register mapping. The moving window region violates that assumption because the same logical address resolves to a different physical register after CALL or RET. The eight global registers do have a fixed mapping, so the RAL block models only those registers.

Each 64-bit global is represented by a `uvm_reg` object with a fixed HDL path to physical registers 128 through 135. The RAL exercise is implemented directly in `rf_ral_test`, not as a separate UVM sequence class. Its build phase constructs the block and sets the DUT HDL root; its run phase loops over all eight globals, writes each with backdoor `poke`, reads it with `peek`, and compares the returned value. The windowed data plane remains under the cycle-accurate reference model.

RAL model and the test that uses it

Fixed-map global registers

```
class rf_globals_block extends uvm_reg_block;
  rf_global_reg g[M];

  virtual function void build();
    default_map = create_map(
      "map", 0, DATA_W/8, UVM_LITTLE_ENDIAN);

    foreach (g[i]) begin
      g[i] = rf_global_reg::type_id::create(
        $sformatf("g%0d", i));
      g[i].configure(this);
      g[i].build();
      default_map.add_reg(
        g[i], i*(DATA_W/8), "RW");
      g[i].add_hdl_path_slice(
        $sformatf("REGISTERS[%0d]", NWIN+i),
        0, DATA_W);
    end
    lock_model();
  endfunction
endclass
```

rf_ral_test: backdoor use

```
function void build_phase(uvm_phase phase);
  env = rf_env::type_id::create("env", this);
  ral = rf_globals_block::type_id::create("ral");
  ral.build();
  ral.add_hdl_path("rf_top.dut_wrap.dut");
endfunction

task run_phase(uvm_phase phase);
  uvm_status_e status;
  uvm_reg_data_t rdata;
  bit [DATA_W-1:0] wdata;

  foreach (ral.g[i]) begin
    wdata = 64'hDEAD_0000 + i;
    ral.g[i].poke(status, wdata);
    ral.g[i].peek(status, rdata);
    if (rdata != wdata)
      'uvm_error("RAL", $sformatf(
        "g[%0d] mismatch", i))
  end
endtask
```

The RAL block covers only physical registers 128–135. The windowed region stays in the cycle-accurate reference model because its logical mapping changes with CWP.

Figure 10: Selected RAL model code and the `rf_ral_test` backdoor *poke/peek* loop.

3.8 Results and coverage closure

Table 7: Windowed register-file verification and synthesis results.

Metric	Result
Scoreboard	1,058 checks, 0 mismatches
Functional coverage	100% (69/69 bins)
Statement coverage	94.73% (36/38)
Branch coverage	94.59% (35/37)
Toggle coverage	100% (438/438)
Assertion coverage	100%
Filtered overall RTL coverage	97.86%
Post-synthesis functional coverage	100% (69/69 bins)
Post-synthesis toggle coverage	38.23% (23,630/61,808)
Clock constraint	3.00 ns
Worst timing slack	1.36 ns, met

The lower gate-level toggle percentage reflects the size and implementation detail of a 136 by 64 mapped register array. Functional completeness is reported separately through the unchanged covergroup, which again reached 69 of 69 bins.

3.9 Code coverage exposes the window-overflow defect

Code coverage revealed the defect. Of 38 RTL statements, the only two not executed were the upward CWP wrap assignment on `CALL` and the downward CWP wrap assignment on `RET`; the corresponding branches were also the only branch misses. Tracing the control flow from those misses showed why. When `CANSAVE` reaches zero, the next `CALL` takes the spill path: `SWP` advances, but `CWP`, `CANSAVE`, and `CANRESTORE` remain unchanged. The mirrored condition occurs on fill. The

wrap assignments are therefore unreachable, and the next procedure continues to address the caller's current window.

Expected window allocation



Observed RTL boundary path



1. Code coverage exposed it: the only RTL misses were the upward and downward CWP wrap assignments. Tracing them showed that spill/fill moves SWP while CWP and the allocation counters remain frozen.

2. Directed testing confirmed it: write a LOCAL register, exhaust the available windows, issue an overflowing CALL, then read the same logical LOCAL. The RTL returns the caller's value instead of the new window's reset value.

Figure 11: Code-coverage trigger and directed confirmation of the window-overflow defect.

The functional consequence was then confirmed with a specification-oriented reference-model mode and a directed test. In that mode, overflow advances CWP together with SWP and underflow retreats it. The test writes a local register, exhausts the available windows, forces an overflowing CALL, and reads the same logical local register. Local registers do not overlap, so the newly selected window should contain its reset value. The RTL instead returns the caller's data. The directed failure confirms the design problem first exposed by the code-coverage holes.

3.10 Post-synthesis findings

Gate-level verification exposed two additional implementation and testbench details:

1. One CANRESTORE bit could remain unknown after power-up because the mapped design used synchronous-only reset logic. The spill path remained correct while the fill path became unknown. Depositing a defined initial state reproduced physical power-up behavior; an asynchronous reset on the small window-management state would remove the simulation ambiguity.
2. The first version of the monitored response fields used two-state types. Sampling an unknown gate-level output converted `x` to zero before it reached the scoreboard. Changing DUT-sampled fields to four-state logic preserved the unknown and allowed the strict comparison to report it correctly.

These findings were invisible in the RTL regression and demonstrate why the same environment was reused at the mapped-netlist level.

4 Conclusion

The P4 environment verifies a registered arithmetic datapath through aligned transactions, an independent arithmetic model, architecture-aware carry coverage, targeted propagation stimulus, reset closure, and post-synthesis reuse. The register-file environment verifies a stateful mapping problem through one-cycle transactions, an explicit-idle driver, a cycle-accurate model of the physical

array and FSM, coverage of the window pointer and counters, directed state-space exploration, and a deliberately limited RAL model for the fixed-map globals.

Both projects reached complete functional coverage at RTL and post synthesis. The separate code-coverage results show different outcomes: complete RTL coverage for the P4 adder, and two unreachable wrap branches in the register file that exposed the overflow defect and led to its directed functional confirmation.

5 Learning resources

The UVM methodology and verification techniques used in these projects were studied through:

- Ray Salemi, *UVM and SystemVerilog training course*, Siemens.
- Ashok B. Mehta, *ASIC/SoC Functional Design Verification: A Comprehensive Guide to Technologies and Methodologies*.